UCS503P - Software Engineering Project

API Gateway

A Self-Service Rate-Limiting Layer for Web APIs

Submitted by

1024030753 Sarthak Jagota
1024030755 Palak Mahajan
1024030758 Aayush Kumar Singh
1024030737 Dushyant Choudhary

Group: XXXX BE Third Year, CSE

Under the guidance of

Dr. Paramveer Sidhu

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **2**
1.1 Background and Context 2
1.1.1 Problem Statement 2
1.2 Project Objectives 3

2 Methodology and Design **4**
2.1 System Architecture 4
2.1.1 High-Level Design 4
2.1.2 Technical Stack 5
2.1.3 Changes from the Proposed Design 6
2.2 Implementation Details 6
2.2.1 Module 1: Gateway and Distributed Rate Limiting 6
2.2.2 Module 2: Persistence, Authentication and Security 8
2.2.3 Module 3: Request Logging and Analytics 9
2.2.4 Module 4: Build, Test and Deployment 9

3 Results and Evaluation **10**
3.1 Testing Methodology 10
3.2 Performance Metrics 10
3.3 Analysis 11

4 Conclusion **14**
4.1 Summary of Work 14
4.2 Key Findings 14
4.3 Future Work and Recommendations 15
4.4 Final Remarks 15

Bibliography **16**

1 Introduction

1.1 Background and Context

Publishing a small web API has become cheap. Platform-as-a-service hosts, managed databases and metered third-party services let a single developer put an API on the internet in minutes. The same pricing model, however, moves the cost of each request from a fixed monthly server bill to a metered, per-request charge. An API that receives more traffic than expected therefore fails in two ways at once: it slows down for everyone, and it costs more.

HTTP itself has no notion of a per-client budget. A client may send requests as fast as it can open connections, and the server learns about the consequences only afterwards. The standard remedy is a *rate-limiting gateway*: a reverse proxy placed between clients and the origin that counts each client's requests and rejects the excess with HTTP 429 *Too Many Requests* [NF12]. Mature products such as Kong [Kon25], AWS API Gateway [Ama25] and Cloudflare provide this, but they are either operationally heavy to run or commercially priced. A developer who only wants to protect one API is otherwise left to build the counting, storage and enforcement logic alone.

This project builds that missing piece: **API Gateway**, a lightweight, self-service gateway that a developer can put in front of an existing API *without modifying the API*. The developer signs up, registers the API's URL, chooses a request budget and an algorithm, and receives a gateway URL and an API key. Clients then call the gateway URL instead of the origin.

The system is deployed and publicly reachable at `https://api-gateway-yp6s.onrender.com/`, and the source code is maintained at `https://github.com/bytewithpalak/API-Gateway-Proje`

1.1.1 Problem Statement

Without per-client accounting, an HTTP API suffers from four recurring failure modes:

- **Resource exhaustion.** One misbehaving client, whether a buggy retry loop, a scraper or a deliberate abuser, can saturate connection pools and worker threads and degrade service for every other client.
- **Uncontrolled cost.** When the API is backed by metered resources, unbounded request volume becomes unbounded spending, discovered only at billing time.
- **No fairness.** A well-behaved client and an abusive one look identical to the server, so capacity goes to whoever sends the most.

- **Expensive remedy.** Existing gateways solve the problem but are heavyweight or paid.

The engineering core of the problem is not the arithmetic of a rate-limiting algorithm, which is well known, but *correctness under concurrency and distribution*. When two gateway instances handle simultaneous requests from the same client, they must never both observe and consume the same remaining slot. A limiter that is correct on one thread but over-admits under concurrent load provides no real protection.

1.2 Project Objectives

The objectives below follow the project proposal submitted on 8 August 2026, and each has a measurable success criterion.

1. **Working reverse proxy with enforcement:** forward requests from a gateway URL to a registered origin, relay the origin's response unchanged, and reject requests over budget with HTTP 429 and a `Retry-After` header [FNR22].

2. **Two selectable algorithms:** implement token bucket and sliding window as per-API policies, with all counter state held in Redis.

3. **Enforcement accuracy (primary metric):** for a limit of N and $M \gg N$ concurrent requests from one client spread over two gateway instances, admit a number of requests within $\pm 2\%$ of N.

4. **Self-service portal:** user signup and login with JWT, API registration with a configurable limit, and an analytics dashboard showing allowed versus blocked traffic, traffic over time, and a request log.

5. **Bounded overhead and reliable logging:** keep the latency the gateway adds at p95 ≤ 50 ms, and record at least 99.9% of requests in the PostgreSQL log with the correct outcome.

6. **Explicit failure behaviour and safe proxying:** define, implement and test what happens when Redis is unreachable, and prevent the gateway from being used as an open proxy into private networks.

7. **Continuous delivery:** build and test every change in CI and deploy the system on free managed infrastructure.

The work was delivered in weekly iterations, starting from a minimal proxy-plus-limiter in week one, and split across three parallel workstreams: gateway and rate limiting, persistence and authentication, and the analytics frontend.

2 Methodology and Design

2.1 System Architecture

The system is a three-tier web application with an additional in-memory tier that holds the rate-limit counters. Figure 2.1 shows the components and the path of a proxied request.

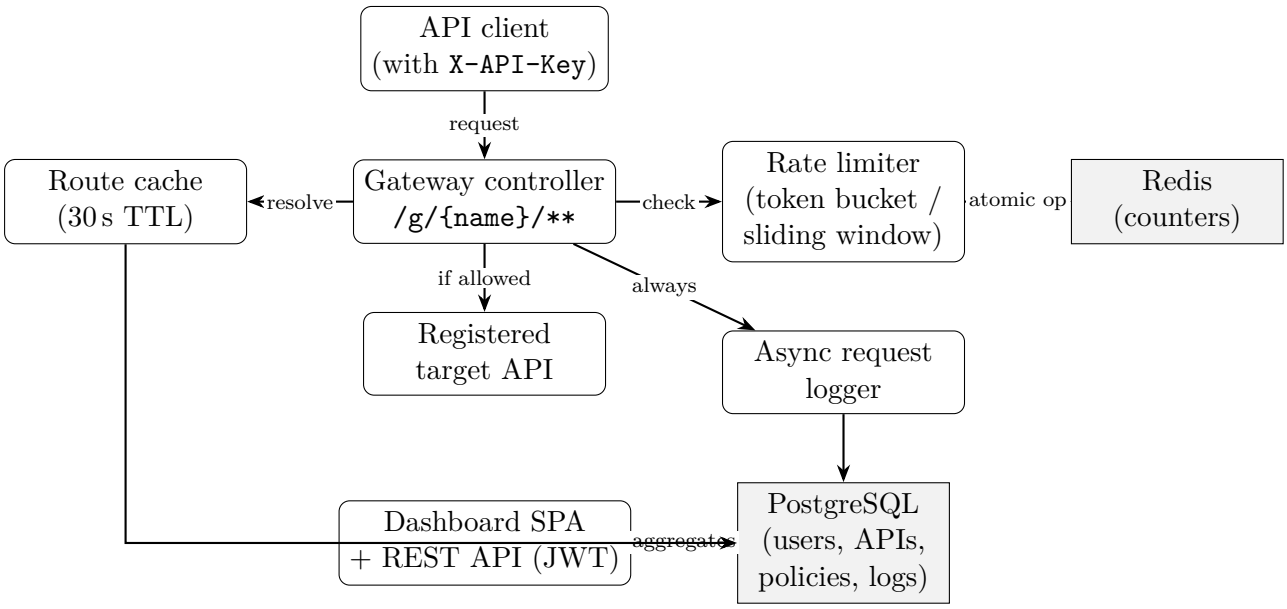


Figure 2.1: System architecture and request path

2.1.1 High-Level Design

The design is organised around four components with separate responsibilities.

Gateway (proxy path). Every request to `/g/{name}/...` is handled by a single controller. It resolves the registered API by its short name, checks the `X-API-Key` header, asks the rate limiter for a decision, and then either forwards the request to the origin or answers with HTTP 429. The remainder of the path after the API name, the query string, the method, the body and all end-to-end headers are forwarded unchanged; hop-by-hop headers (such as `Connection` and `Transfer-Encoding`) and the gateway's own API key are stripped. Redirects from the origin are relayed to the client rather than followed.

Rate limiter and counter store. All counting state lives in Redis, so the gateway itself is stateless and any number of instances can share one budget. Each decision is

made by a single atomic operation inside Redis, which is what prevents two instances from admitting the same slot.

Management API and persistence. A REST API, protected by JWT authentication, lets users register APIs, edit their policies, rotate keys and delete APIs. PostgreSQL stores users, registered APIs, policies and the request log. The request log is written asynchronously, outside the client's request path.

Dashboard. A single-page web application, served by the same Spring Boot service, reads pre-aggregated statistics over REST and renders charts, the request log, a test lab and a system health page. Aggregation happens in the database, so raw logs are never shipped to the browser.

The core request workflow is:

1. A developer signs up, logs in, and registers a target API, receiving a gateway URL and an API key.
2. A client sends a request to the gateway URL with the key in the `X-API-Key` header.
3. The gateway resolves the policy for the API and performs an atomic consume-or-reject operation in Redis, keyed by API and client IP.
4. If allowed, the request is proxied and the origin's response is relayed verbatim; otherwise the gateway returns HTTP 429 with `Retry-After`.
5. In both cases an outcome record is written to PostgreSQL in the background.
6. The dashboard reads aggregated records over REST and renders them.

2.1.2 Technical Stack

- **Framework/Language:** Java 17, Spring Boot 4.1 (Spring Web MVC, Spring Data JPA, Spring Security, Bean Validation) [VMw26].
- **Database:** PostgreSQL 16 for durable data; Redis 7 for ephemeral rate-limit counters [Red25].
- **Libraries:** Bucket4j 8.19 with its Jedis integration for the distributed token bucket [Buc25]; Jedis 5.2 as the Redis client; JJWT 0.12 for JSON Web Tokens [JBS15]; BCrypt (via Spring Security) for password hashing [PM99]; the JDK `HttpClient` for outbound proxy calls.
- **Frontend:** a dependency-free single-page application in HTML, CSS and JavaScript, with charts drawn as inline SVG.

- **Testing:** JUnit 5 and Testcontainers 1.20, which start real Redis and PostgreSQL containers for integration tests [Tes25].
- **Tools and deployment:** Maven, Docker and Docker Compose, GitHub Actions for CI, Render for hosting (behind Cloudflare), and UptimeRobot for availability monitoring.

2.1.3 Changes from the Proposed Design

Two parts of the proposed technology stack were replaced during development. Both were working in early iterations and were changed deliberately.

- **Spring Cloud Gateway → custom proxy controller.** Iterations one to three used Spring Cloud Gateway (MVC variant) with routes declared in configuration. Once APIs were registered by users at runtime and stored in PostgreSQL, every route had to be resolved per request from the database, with its own key and policy. The team had also hit a filter-ordering problem in week one, where the gateway's internal URL-resolution filter ran before our custom filter. A small dedicated controller with the JDK `HttpClient` made the request path explicit, removed the version coupling between Spring Boot and the Spring Cloud release train, and gave direct control over header filtering, timeouts and latency measurement.
- **React + Recharts on Vercel → single static page served by the backend.** A React dashboard with Recharts was built in an earlier iteration. It was later replaced by a single static page served by the Spring Boot application, so that the whole system ships as one container with one URL and no separate frontend build or hosting. The page can still be hosted elsewhere, because the backend's CORS configuration allows any origin and authentication uses a bearer header rather than cookies.

2.2 Implementation Details

2.2.1 Module 1: Gateway and Distributed Rate Limiting

Token bucket. A bucket of capacity C is refilled at rate $r = N/W$ tokens per second, where N is the configured limit and W the window in seconds. Each request consumes one token; a request arriving at an empty bucket is rejected. This allows controlled bursts up to C while keeping the long-run average at r. The capacity defaults to N and can be set separately as a burst capacity. The bucket is implemented with Bucket4j's Redis proxy manager, which uses compare-and-swap updates so that concurrent consumers from different instances cannot both take the last token. The policy values are part of

the Redis key, so editing a policy starts fresh buckets rather than reusing ones built with the old numbers. Idle buckets expire automatically once they would be full again.

Sliding window. An early version used Bucket4j's interval refill, but this behaves as a *fixed* window: a client can send N requests at the end of one window and N more at the start of the next, reaching $2N$ in a short span. It was replaced with a true sliding window (a "sliding log"), implemented as one Lua script that Redis executes atomically (Listing 2.1). Each admitted request is stored as a timestamp in a sorted set; entries older than the window are removed, and a request is admitted only if fewer than N remain. The script uses Redis's own clock, so differences between gateway instances' clocks cannot affect the result. When a request is rejected, the script returns the exact time until the oldest entry leaves the window, which becomes the `Retry-After` value.

Listing 2.1: Atomic sliding-window check in Redis (Lua)

```
local key    = KEYS[1]
local window = tonumber(ARGV[1])
local limit  = tonumber(ARGV[2])
local t      = redis.call('TIME')
local now    = tonumber(t[1]) * 1000 + math.floor(tonumber(t[2]) / 1000)
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
if redis.call('ZCARD', key) < limit then
  redis.call('ZADD', key, now, ARGV[3])
  redis.call('PEXPIRE', key, window)
  return -1
end
local oldest = redis.call('ZRANGE', key, 0, 0, 'WITHSCORES')
return tonumber(oldest[2]) + window - now
```

Retry-After. For both algorithms the gateway reports the real time until the next request would succeed, rounded up to whole seconds, rather than the length of the whole window.

Failure policy when Redis is unreachable. This is an explicit, configurable decision. *Fail-open* (the default) admits traffic without limits, preserving the API's availability; *fail-closed* rejects traffic, preserving protection. The default is fail-open because a Redis outage that took every protected API offline would be worse, for most users, than a short period without limiting. Redis calls use a 500 ms socket timeout, and after one failure the limiter skips Redis for five seconds (a minimal circuit breaker), so an outage does not add a timeout to every request. The policy can be changed from the dashboard's System page, which also shows whether Redis is currently up.

Client identity. Budgets are kept per client IP address. The production deployment runs behind Cloudflare, where the socket address is a Cloudflare edge server that changes between requests; this initially spread one client's requests over many buckets. The gateway can therefore be configured to read the real client IP from the `CF-Connecting-IP` header. The option is off by default, because trusting such a header on a directly reachable server would let clients choose their own identity.

Route cache. Looking up the API and policy in PostgreSQL on every request would add database latency to the proxy path. Routes are cached for 30 seconds; edits made from the dashboard evict the entry immediately on the instance that handled them and take effect on other instances within 30 seconds.

2.2.2 Module 2: Persistence, Authentication and Security

Data model. Four entities are stored in PostgreSQL: `users` (email and BCrypt hash), `registered_apis` (owner, name, unique short name, target URL, unique API key, active flag), `policies` (one per API: algorithm, limit, window, optional burst capacity) and `request_logs` (API, client identity, outcome, HTTP status, gateway-added latency, timestamp). The request log has a composite index on `(api_id, timestamp)`, which matches every dashboard query.

Authentication. Users sign up and log in with an email and password; passwords are hashed with BCrypt and never stored in plain text. A successful login returns a signed JWT that the dashboard sends as a bearer token. The API is stateless (no server sessions), and an expired or missing token returns HTTP 401 so the dashboard can send the user back to the login page. Every management and dashboard endpoint returns only the caller's own APIs, and requests for another user's API return 404 rather than 403, so API identifiers are not revealed.

Proxy-abuse (SSRF) protection. A gateway that forwards to arbitrary URLs could be abused to reach internal services or cloud metadata endpoints [OWA24]. The gateway only forwards to targets registered by an authenticated owner, and each target URL is validated: it must use `http` or `https`, and *every* address its host resolves to must be public. Loopback, link-local (including the `169.254.169.254` metadata address), private, any-local, multicast and IPv6 unique-local addresses are refused. The check runs at registration and again whenever a route is reloaded, so a domain later pointed at an internal address stops working.

Secrets. Database credentials, the Redis URL and the JWT signing key are read from environment variables configured on the hosting platform, not from the repository. API

keys are random UUIDs and can be rotated from the dashboard; the old key stops working immediately.

2.2.3 Module 3: Request Logging and Analytics

Asynchronous logging. After each decision the gateway calls an `@Async` logging service, so the database write happens on a separate thread pool and never adds to the client's response time. The trade-off is that a crash between sending the response and writing the row loses that row; the log-completeness metric exists to measure this risk.

Gateway-added latency. For each proxied request the gateway records its total handling time minus the time spent waiting for the origin. This isolates the overhead the gateway itself adds, which is the quantity the latency objective concerns.

Dashboard endpoints. All aggregation is done in PostgreSQL: allowed versus blocked counts per API over the last 24 hours, 24 hourly traffic buckets for the chart, p50 and p95 gateway-added latency (using `percentile_cont`), recent throughput, and a paginated request log filterable by outcome and API. Page sizes are capped at 100 rows.

Dashboard pages. The single-page application provides: an *overview* with totals and a 24-hour traffic chart; *My APIs* for registering, editing, rotating keys and deleting APIs, with a ready-to-copy `curl` example; *Request logs* with filters and pagination; a *Test lab* that fires a configurable number of concurrent requests at one of the user's gateway URLs and reports admitted, blocked and deviation from the limit, plus a simulated side-by-side comparison of the two algorithms; and a *System* page with Redis health, the fail policy and latency figures. The layout is responsive and works on phone-width screens.

2.2.4 Module 4: Build, Test and Deployment

A GitHub Actions workflow runs `mvn verify` on every push and pull request, which compiles the application and runs the full integration test suite against real Redis and PostgreSQL containers. A multi-stage Dockerfile builds the application with Maven and packages it on a slim Java 17 runtime image. Docker Compose starts the whole system (application, PostgreSQL and Redis) locally with one command. In production the container runs on Render, which redeploys automatically when commits reach the main branch, and UptimeRobot checks the site periodically, which also keeps the free-tier instance from sleeping.

3 Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** Rate-limiter behaviour, including the Redis failure path, is tested directly against the `RateLimiter` class, without starting the web application. Repository queries are tested in isolation for each filter.
- **Integration Testing:** All automated tests run against *real* Redis 7 and PostgreSQL 16 instances started by Testcontainers, not in-memory substitutes, because the behaviour being tested (atomic Redis operations, the async thread pool, PostgreSQL's connection pool) differs in ways that matter. The suite runs in CI on every push.
- **Concurrency and multi-instance testing:** The accuracy test creates two independent rate-limiter instances, each with its own Redis connection pool, exactly as two application servers would have. It sends 400 requests from 32 threads for a single client, alternating between the two instances, against a limit of $N = 20$ with a one-hour window (so no refill happens during the test), for both algorithms. Earlier, a manual test also ran two full gateway processes on ports 8080 and 8081 against one Redis and alternated requests between them.
- **Performance Testing:** Gateway overhead was benchmarked by comparing 20 requests sent directly to an origin (httpbin) with 20 sent through the gateway, using persistent connections on both sides. In production, every request's gateway-added latency is logged and the p50 and p95 are shown on the System page.
- **Manual and API testing:** Endpoints were exercised with `curl`, Postman and the browser, including refill behaviour, stopping Redis while the gateway was running, and CORS behaviour from a browser origin.
- **User Testing:** A structured usability study with external developers has not yet been carried out (see Section 3.3).

3.2 Performance Metrics

Table 3.1 lists the automated test suite, run on 15 October 2026 with Docker and Testcontainers. All eight tests passed.

Table 3.2 compares the measured results with the targets set in the proposal.

Test	What it verifies	Result
`tokenBucketIsExact...`	Token bucket, 2 instances, 400 requests, $N = 20$	20 admitted
`slidingWindowIsExact...`	Sliding window, 2 instances, 400 requests, $N = 20$	20 admitted
`failOpenAndFailClosed...`	Redis down: open allows, closed blocks, breaker answers in < 50 ms	Pass
`concurrentAsyncLogging...`	2000 concurrent async log writes reach PostgreSQL	2000 / 2000
`filtersByOutcome`	Log filter by outcome	Pass
`filtersByClientIdentity`	Log filter by client	Pass
`filtersByDateRange`	Log filter by date range	Pass
`neverLeaksAcrossApiId`	No rows from another API	Pass

Table 3.1: Automated test suite results (8 tests, 0 failures)

3.3 Analysis

Objectives met. The primary objective, enforcement accuracy under concurrent load across two instances, was met exactly: both algorithms admitted precisely N requests out of $20N$, which is better than the $\pm 2\%$ target. This confirms that performing each decision as one atomic operation inside Redis (compare-and-swap for the token bucket, a single Lua script for the sliding window) removes the race condition the project set out to address. Log completeness was 100% under 2000 concurrent writes, and the Redis failure path behaves as configured. Objectives 1, 2, 4 and 6 are fully implemented, and the system is deployed and publicly reachable.

Partly met. The local latency benchmark (≈ 26.5 ms added) is well inside the 50 ms budget, but it is a mean over 20 requests, not a p95 on the deployed system. The deployed system now records gateway-added latency for every request, so the p95 can be reported directly from the System page after a measurement run. The two-instance accuracy test runs both limiter instances in one test process against one real Redis; a run against two deployed instances, with requests counted at the origin, would make the evidence stronger. CI builds and tests every change, but linting, container image publishing and a separate staging environment are not yet part of the pipeline.

Not yet done. The usability study with 5–10 developers, measurement of dashboard freshness and sustained throughput per instance count, and the stretch goal of request replay remain open. The algorithm comparison in the Test lab is a client-side simulation rather than a live run of both algorithms under identical load.

Challenges and how they were resolved.

Metric	Target	Achieved
Enforcement accuracy, 2 instances, $M = 20N$	within $\pm 2\%$ of N	20/20 admitted for both algorithms (0% deviation)
Enforcement accuracy, single instance (week 3)	within $\pm 2\%$ of N	20 of 200 admitted, $N = 20$ (0% deviation)
Shared budget across two running gateways (manual)	one shared budget	budget shared; both ports returned 429 once it was spent
Gateway-added latency	p95 ≤ 50 ms	≈ 26.5 ms mean over 20 requests (local benchmark); deployed p95 not yet recorded
Log completeness	$\geq 99.9\%$	100% (2000 of 2000 concurrent writes)
Redis failure handling	defined and tested	fail-open and fail-closed both verified; breaker responds in under 50 ms
Availability	$\geq 99\%$	monitored with UptimeRobot; figure for the pilot window not yet reported

Table 3.2: Measured results against proposal targets

- **Stale bucket state.** Bucket4j applies a bucket's configuration only when the bucket is first created in Redis, so policy edits were silently ignored. Including the policy values in the Redis key fixed this: an edited policy starts new buckets.
- **Fixed window instead of sliding window.** The first "sliding window" used interval refill, which allows up to $2N$ requests around a window boundary. It was replaced with the atomic sliding-log Lua script.
- **Wrong client identity behind Cloudflare.** In production, one client appeared as many different Cloudflare edge IPs and received many budgets. Reading `CF-Connecting-IP`, behind a configuration switch, fixed this without making local deployments spoofable.
- **Misleading latency benchmark.** The first benchmark showed the gateway as 760 ms *faster* than calling the origin directly, because each direct `curl` call paid a fresh TLS handshake while the gateway reused a warm connection. Using persistent connections on both sides gave the realistic 26.5 ms figure.
- **Testcontainers and Docker API mismatch.** Recent Docker Desktop versions rejected the client bundled with Testcontainers 1.20. Pinning the Docker API version in `docker-java.properties` resolved it.
- **Browser-only failures.** Requests that worked in `curl` failed in the browser because of CORS preflight rules and the browser's native login pop-up on HTTP 401.

The CORS configuration was corrected and unauthenticated responses no longer trigger the pop-up, which showed that `curl` tests alone do not cover browser clients.

4 Conclusion

4.1 Summary of Work

The project delivered API Gateway, a deployed, self-service rate-limiting reverse proxy that protects an existing API without modifying it. A developer can sign up, register an API, choose between token bucket and sliding window, and receive a gateway URL and key; clients over budget receive HTTP 429 with an accurate `Retry-After`. Counters live in Redis and are updated atomically, so the limit holds across gateway instances. Every request is logged asynchronously to PostgreSQL and summarised on a responsive dashboard with traffic charts, a filterable request log, a load-testing lab and a system health page. The system is protected against SSRF, has an explicit and tested Redis failure policy, is tested in CI against real Redis and PostgreSQL, and runs as a single Docker container on Render.

Of the objectives in Section 1.2, the proxy and enforcement, both algorithms, the primary accuracy metric, the self-service portal, failure handling and safe proxying, and CI with deployment have been met. The latency target is met on the local benchmark but still needs a p95 measured on the deployed system.

4.2 Key Findings

- **Finding 1:** Atomicity, not the algorithm, decides correctness. With every decision made by a single atomic Redis operation, two independent instances admitted exactly N of $20N$ concurrent requests for both algorithms.
- **Finding 2:** Library behaviour can quietly change the algorithm. Bucket4j's interval refill, used as a "sliding window", actually behaves as a fixed window, and Bucket4j keeps the configuration a bucket was created with. Both were found only by testing the system's real behaviour rather than trusting its configuration.
- **Finding 3:** Deployment details affect correctness. Behind a CDN, the socket address is not the client, which silently broke per-client limits in production until the real client IP was read from a trusted header.
- **Finding 4:** Measurement method matters as much as the system. Comparing cold and warm connections made the gateway look 760 ms faster than the origin; only a like-for-like benchmark gave a meaningful overhead figure.

4.3 Future Work and Recommendations

1. **Rate-limit login attempts** to slow down password guessing, reusing the gateway's own Redis-backed limiter.
2. **Store API keys hashed** rather than in plain text, showing each key to the owner only once when it is created or rotated.
3. **Close the DNS-rebinding gap:** target addresses are already re-checked whenever a route is reloaded, but the address the proxy actually connects to should also be checked at connection time.
4. **Optional per-API-key budgets:** let an owner choose one shared budget per API key instead of a separate budget for each client IP.
5. **Alerting:** Redis health and latency are already shown on the System page; add alerts (for example by email) when Redis goes down or p95 latency crosses the budget.
6. **More tests** for authentication, authorization and tenant isolation, checking that one user can never see or change another user's APIs or logs.
7. **Complete the evaluation** with a load test against the deployed system, reporting p95 overhead and uptime over a fixed period.

4.4 Final Remarks

The project showed that the hard part of rate limiting is not the formula but keeping it correct when many requests arrive at once on more than one server, and when the system runs behind real infrastructure such as a CDN. Building on mature components (Spring Boot, Redis, Bucket4j, PostgreSQL) let the team spend its effort on that correctness, on testing it against real services, and on measuring it honestly. The result is a working, deployed gateway that a developer can put in front of an API in a few minutes, and a clear list of the measurements that remain to complete the evaluation.

Bibliography

[Ama25] Amazon Web Services. *Throttle Requests to Your REST APIs for Better Throughput in API Gateway*. 2025. URL: `https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-request-throttling.html` (visited on 10/15/2026) (cit. on p. 2).

[Buc25] Bucket4j contributors. *Bucket4j: Java Rate-Limiting Library Based on Token-Bucket Algorithm*. 2025. URL: `https://bucket4j.com/` (visited on 10/15/2026) (cit. on p. 5).

[FNR22] Roy T. Fielding, Mark Nottingham, and Julian Reschke. *HTTP Semantics*. RFC 9110. Internet Engineering Task Force, 2022 (cit. on p. 3).

[JBS15] Michael B. Jones, John Bradley, and Nat Sakimura. *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force, 2015 (cit. on p. 5).

[Kon25] Kong Inc. *Kong Gateway Documentation*. 2025. URL: `https://docs.konghq.com/gateway/` (visited on 10/15/2026) (cit. on p. 2).

[NF12] Mark Nottingham and Roy T. Fielding. *Additional HTTP Status Codes*. RFC 6585. Internet Engineering Task Force, 2012 (cit. on p. 2).

[OWA24] OWASP Foundation. *Server-Side Request Forgery Prevention Cheat Sheet*. 2024. URL: `https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html` (visited on 10/15/2026) (cit. on p. 8).

[PM99] Niels Provos and David Mazières. "A Future-Adaptable Password Scheme". In: *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*. 1999, pp. 81–91 (cit. on p. 5).

[Red25] Redis Ltd. *Redis Documentation: Scripting with Lua and Sorted Sets*. 2025. URL: `https://redis.io/docs/` (visited on 10/15/2026) (cit. on p. 5).

[Tes25] Testcontainers contributors. *Testcontainers for Java*. 2025. URL: `https://java.testcontainers.org/` (visited on 10/15/2026) (cit. on p. 6).

[VMw26] VMware Tanzu. *Spring Boot Reference Documentation*. 2026. URL: `https://docs.spring.io/spring-boot/` (visited on 10/15/2026) (cit. on p. 5).